Understanding Missing Values

What are Missing Values?

Missing values are observations where **no value has been recorded** for one or more variables in a dataset.

In Pandas, missing values are commonly represented as:

- `NaN` (Not a Number)
- `None`
- `NaT` (Not a Time) for datetime columns

Example:

Employee_ID	Name	Age	Salary	Department
101	Alice	25	50000	HR
102	Bob	NaN	65000	IT
103	Charlie	31	NaN	Finance
104	David	28	72000	NaN

Why Do Missing Values Occur?

Missing values occur for many reasons in real-world datasets.

1. Human Error

People may skip fields while entering information.

Example:

A customer registration form asks for:

- Name
- Email
- Phone Number
- Annual Income

The customer chooses not to enter Annual Income.

2. Data Entry Mistakes

Information may accidentally be left blank.

Example:

Hospital receptionist forgets to enter the patient's blood group.

3. Sensor Failure

IoT devices or sensors sometimes stop recording data.

Example:

Temperature recorded every hour.

4. System Errors

Software crashes or database failures can produce missing records.

Example:

During payment processing, the payment status was not saved.

5. Data Merging Issues

When combining datasets, some records may not have matching values.

6. Privacy Reasons

Some people intentionally do not provide sensitive information.

Examples:

- Salary
- Phone Number

- Religion
- Income
- Medical History

Why are Missing Values Important?

Ignoring missing values can lead to incorrect conclusions.

Suppose employee salaries are:

```
None
50000
60000
70000
NaN
80000
```

If the missing value is ignored incorrectly, the calculated average may not represent the true population.

Missing values can affect:

- Mean
- Median
- Standard deviation

- Correlation
- Data visualization
- Machine learning predictions

Impact of Missing Values

1. Reduced Dataset Size

Dropping missing values may remove a large number of records.

2. Biased Analysis

Suppose only high-income customers skipped the income field.

Average income calculated after dropping those rows will be lower than reality.

Business decisions become inaccurate.

3. Poor Machine Learning Performance

Most machine learning algorithms cannot work with missing values directly.

Before Handling Missing Values

Before applying any technique, ask these questions:

1. Which columns contain missing values?
2. How many missing values are there?
3. What percentage of the data is missing?
4. Is the missing data numerical or categorical?
5. Is the missingness random or systematic?
6. Is the column important for the analysis?
7. Should the values be removed or imputed?

Common Strategies (Overview)

Technique	When to Use
Drop Rows	Very few missing records
Drop Columns	Column has too many missing values and low importance
Mean Imputation	Normally distributed numerical data
Median Imputation	Skewed numerical data or outliers present
Mode Imputation	Categorical variables

Forward Fill	Time-series data
Backward Fill	Sequential data
KNN Imputation	When relationships among features can improve estimates

Best Practices

- Never ignore missing values.
- Analyze the pattern before deciding how to handle them.
- Consider the business context before choosing an imputation strategy.
- Document every decision made during data cleaning.
- Validate the dataset after handling missing values to ensure no unintended changes were introduced.

MCAR vs MAR vs MNAR

Type	Meaning	Example	Can we impute?
MCAR (Missing Completely At Random)	Missingness is completely random and unrelated to any variable.	A sensor randomly fails to record temperature.	✅ Yes. Simple methods (mean, median, mode) usually work well.
MAR (Missing At Random)	Missingness depends on **other observed variables**, but not on the missing value itself.	Income is more likely to be missing for younger people, but age is known.	✅ Yes. Prefer advanced methods (group-wise imputation, KNN, MICE, regression).
MNAR (Missing Not At Random)	Missingness depends on the **missing value itself**.	High-income individuals choose not to disclose their income.	⚠️ Difficult. Simple imputation introduces bias. Domain knowledge or specialized statistical models are often needed.

Scenario 1

A hospital's blood pressure machine randomly malfunctioned for a few hours, so blood pressure readings are missing for some patients.

Scenario 2

A survey found that younger respondents were much more likely to skip the "Annual Income" question than older respondents.

Scenario 3

People with very high salaries intentionally chose not to disclose their income.

Scenario 4

During data entry, 5% of customer phone numbers were lost because the database server crashed.

Scenario 5

Customers who placed online orders were less likely to provide their phone number, but you know whether the order was online or offline.

Scenario 6

Students with very low exam marks decided not to submit their answer sheets.

Scenario 7

A rain sensor stopped working every Sunday due to scheduled maintenance.

Scenario 8

A few pages of printed questionnaires were accidentally damaged during transportation.

Easy Memory Trick

- 🎲 **MCAR → Random chance** (no pattern)
- 🔍 **MAR → Missing because of another known column**
- 🔒 **MNAR → Missing because of its own value** (the missing value itself influences whether it is missing

How do we decide to drop a column?

A common rule of thumb is:

Missing %	Decision
< 5%	Usually impute or ignore
5–30%	Analyze carefully; imputation often works
30–60%	Depends on business importance
> 60–70%	Consider dropping the column (unless it's business-critical)

When should we use which?

Method	Data Type	Outliers?	Best Used For
Mean	Numerical	❌ No	Normally distributed data without significant outliers
Median	Numerical	✅ Yes	Skewed distributions or data with outliers
Mode	Categorical / Numerical	Doesn't matter	Categorical features or the most common value

Column	Missing	Which method?
Age (contains 120)	2	?
Salary	2	?
Department	2	?
Gender	1	?

KNN Imputation (K-Nearest Neighbors Imputation)

KNN Imputation is an **advanced imputation technique** where missing values are filled using the values of **similar data points (neighbors)** instead of using a global statistic like the mean or median.

It is available in **scikit-learn** through:

```
None
from sklearn.impute import KNNImputer
```

Why do we need KNN Imputation?

Suppose we have the following employee data.

Emp_ID	Age	Experience	Salary
101	25	2	50000
102	28	4	60000
103	26	3	NaN
104	45	20	120000
105	47	22	125000

Question:

What should be the salary of Employee 103?

Using Mean Imputation:

```
None
Mean Salary

= (50000+60000+120000+125000)/4

= 88,750
```

Would ₹88,750 make sense?

No!

Employee 103 is much more similar to Employees 101 and 102 than to Employees 104 and 105.

KNN recognizes this similarity.

Basic Idea

Instead of asking

> "What is the average salary?"

KNN asks

> "Who are the most similar employees?"

Then it uses those employees to estimate the missing value.

Step-by-Step Working

Dataset

Age	Experience	Salary
25	2	50000
28	4	60000
26	3	NaN
45	20	120000
47	22	125000

Missing salary for

```
None
Age = 26
Experience = 3
```

Step 1

Ignore rows with missing target value.

Known rows

```
None
25   2   50000

28   4   60000

45   20 120000

47   22 125000
```

Step 2

Compute distance between Employee 103 and every other employee.

Usually Euclidean Distance is used.

Distance from

```
(26,3)
```

to

```
(25,2)
```

```
√((26−25)²+(3−2)²)

= √2

≈1.41
```

Distance to

None
```
(28,4)
```

None
```
≈2.24
```

Distance to

None
```
(45,20)

≈25.5
```

Distance to

None
```
(47,22)

≈28
```

Step 3

Choose K nearest neighbors.

Suppose

```
None
k=2
```

Nearest neighbors are

```
None
Employee 101

Employee 102
```

Step 4

Take average salary

```
None
(50000+60000)/2
```

```
=55000
```

Missing salary becomes

```
None
55000
```

Notice

Mean Imputation

```
None
88750
```

KNN

```
None
55000
```

Much more realistic.

Advantages

✅ Better than Mean/Median for many datasets.

✅ Preserves relationships between features.

✅ Uses neighboring observations rather than a single global statistic.

✅ Works well when similar records exist.

Limitations

❌ Computationally expensive for large datasets.

❌ Sensitive to feature scales. Features should usually be scaled first (e.g., using `StandardScaler` or `MinMaxScaler`) because distance calculations are affected by the magnitude of each feature.

❌ Choosing the wrong value of `k` can reduce accuracy.

❌ Performance degrades if many features or many rows have missing values.

When Should We Use KNN?

Situation	Recommendation
Small to medium dataset	✅ Good choice
Numerical features	✅ Excellent
Strong relationships among features	✅ Recommended
Large dataset (millions of rows)	❌ Can be slow
Categorical features only	❌ Not ideal; mode or other categorical imputation methods are often more appropriate

Comparison

Method	Uses	Considers Other Features?	Outlier Sensitive?	Best For
Mean	Average	❌ No	✅ Yes	Symmetric numerical data
Median	Middle value	❌ No	❌ No	Skewed numerical data
Mode	Most frequent value	❌ No	❌ No	Categorical data
Forward Fill	Previous value	❌ No	❌ No	Time-series data
Backward Fill	Next value	❌ No	❌ No	Time-series data
KNN	Similar records	✅ Yes	Depends on neighbors and scaling	Numerical data with correlated features